MATERIAL EXTRA

Arquitetura REST

growdev

INTRODUÇÃO AO REST

A arquitetura REST (*Representational State Transfer*) é um estilo de design para a construção de APIs (*Application Programming Interfaces*) que utiliza os princípios do protocolo HTTP para permitir a comunicação entre sistemas. REST não é um padrão oficial ou tecnologia específica, mas uma abordagem que promove a simplicidade, escalabilidade e reutilização de componentes. O objetivo principal é representar recursos do sistema (como usuários, produtos ou pedidos) através de URLs, fornecendo operações bem definidas para interagir com esses recursos.

Os sistemas RESTful são **sem estado (stateless)**, o que significa que cada requisição enviada ao servidor deve conter todas as informações necessárias para processar a operação. Essa independência permite que as APIs REST sejam altamente escaláveis e integráveis em diversos contextos.

HTTP e HTTPS

HTTP (*Hypertext Transfer Protocol*) é um protocolo de comunicação usado na web para transferir informações entre clientes (como navegadores) e servidores. Ele é a base da comunicação na internet e segue um modelo de **requisição** e **resposta**, onde o cliente (geralmente um navegador) faz uma solicitação ao servidor, e o servidor responde com os dados pedidos ou mensagens de erro.
A principal diferença entre **HTTP** e **HTTPS** é a segurança. Enquanto o HTTP transmite dados de forma simples (não criptografada), o HTTPS utiliza criptografia para proteger a comunicação entre cliente e servidor.

Métodos/Verbos HTTP

Os métodos HTTP são as ações que podem ser realizadas sobre os recursos representados pelas URLs. Cada método tem um propósito específico:

- **GET**: Obter informações de um recurso.
- **POST**: Criar um novo recurso.
- **PUT**: Atualizar completamente um recurso existente.
- **PATCH**: Atualizar parcialmente um recurso.
- **DELETE**: Remover um recurso.

Esses métodos permitem que as APIs RESTful sigam convenções padronizadas, tornando-as mais fáceis de entender e usar.

Verbo GET

O verbo GET é utilizado para recuperar informações de um servidor. Ele não altera os dados do recurso, sendo considerado um método seguro e idempotente. Isso significa que, independentemente de quantas vezes seja chamado, o resultado será sempre o mesmo, desde que os dados no servidor permaneçam inalterados (leia mais sobre em Idempotente - Glossário do MDN Web Docs: Definições de termos relacionados à Web | MDN).

Verbo POST

O verbo POST é comumente utilizado para criar novos recursos no servidor. Quando você envia dados para o servidor com um POST, ele geralmente cria um novo recurso com base nesses dados.

O POST é considerado não idempotente, ou seja, se você enviar a mesma requisição várias vezes, o resultado pode ser diferente a cada envio (como a criação de múltiplos recursos). Isso é diferente de métodos como GET, que são idempotentes (um GET repetido sempre trará o mesmo resultado sem alterações no servidor).

Verbo PUT e PATCH

Os verbos PUT e PATCH são métodos HTTP usados em APIs REST para **atualizar recursos** no servidor, mas eles têm diferenças importantes em termos de como essa atualização é realizada.

➔ **PUT**

- **Objetivo**: Substituir completamente um recurso existente no servidor.
- **Comportamento**: Quando você usa o PUT, você envia uma versão completa do recurso que deseja atualizar. O servidor substitui o recurso existente por esse novo estado.
- **Idempotência**: O PUT é idempotente, ou seja, se você enviar a mesma requisição várias vezes, o resultado será o mesmo, sem criar recursos adicionais ou alterações inesperadas.
- **Exemplo**: Se você tem um recurso de usuário com o ID 123, e deseja atualizar todas as informações dele, você enviaria a requisição PUT com todos os dados do usuário (não apenas os dados alterados).

➔ **PATCH**

- **Objetivo**: Atualizar parcialmente um recurso existente no servidor.
- **Comportamento**: O PATCH permite que você envie apenas os dados que deseja alterar, sem precisar fornecer o recurso

completo. O servidor aplica as mudanças parciais aos dados existentes.

- **Idempotência**: O PATCH também pode ser idempotente, mas não é garantido. A idempotência do PATCH depende de como o servidor é configurado para processar as atualizações parciais.
- **Exemplo**: Se você deseja alterar apenas o campo "email" de um usuário, pode enviar uma requisição PATCH com apenas o dado que foi alterado.

Verbo DELETE

O verbo DELETE é um método HTTP usado em APIs REST para remover recursos do servidor. É uma das operações fundamentais no design de APIs RESTful e faz parte do conjunto de métodos para manipular recursos (junto com GET, POST, PUT e PATCH).

O DELETE é considerado idempotente, o que significa que fazer a mesma requisição múltiplas vezes não mudará o estado do servidor após a exclusão inicial. Por exemplo, na primeira requisição, o recurso será excluído e em requisições subsequentes, o recurso já não estará presente, mas a resposta será consistente (geralmente um status de erro ou confirmação de que o recurso já não existe).

URL

Na arquitetura REST, a URL (*Uniform Resource Locator*) é usada para identificar os recursos disponíveis em uma API. Ela é composta por diferentes partes que determinam como o cliente se comunica com o servidor.

https://www.api.com:8080/usuario/recurso?pesquisa=valor

- Protocolo
- Hostname
- Porta
- Rota
- Parâmetro

- **Protocolo:** Protocolo de comunicação usado (HTTP ou HTTPS).
- **Hostname:** Refere-se ao nome de um dispositivo em uma rede. Pode ou não estar associado a um domínio.
- **Porta:** É um número lógico associado a um protocolo ou serviço específico em um dispositivo. Permitem que os dados sejam enviados e recebidos para o aplicativo ou serviço correto em um computador, mesmo que ele esteja executando várias operações ao mesmo tempo.
- **Rota:** O caminho adicional que especifica o recurso ou a operação desejada.
- **Parâmetro:** São informações passadas na própria URL para fornecer dados adicionais a um servidor ou aplicação. Esses parâmetros geralmente seguem um formato padrão e são usados para filtrar, personalizar ou transmitir dados.

Com uma URL bem definida, a construção de endpoints específicos para acessar e manipular recursos se torna simples, padronizada e intuitiva. Para isso, devemos seguir algumas boas práticas básicas:

- **Usar substantivos no plural**: Representar os recursos da API de forma genérica e descritiva.
  Exemplo: `/users` e não `/user`

- **Identificação por ID:** Para acessar, atualizar ou excluir um item específico, usa-se o ID ou outro identificador único.
  Exemplo: `/users/:id` onde id pode ser qualquer identificador único.

- **Hierarquia clara:** Para recursos relacionados, pode-se usar rotas hierárquicas.
  Exemplo: `/users/:id/orders` para acessar os pedidos de um usuário específico.

- **Evitar o uso de verbos:** A operação (como criar ou deletar) deve ser indicada pelo método HTTP, não pela rota.
  Exemplo: `/users` e não `/getUsers`

- **Uso correto dos verbos HTTP**: Deve-se sempre respeitar a aplicação dos verbos HTTP. Ou seja, uma rota GET não pode ser usada para criar um novo recurso.

Rotas na arquitetura REST

As rotas combinam URLs com os métodos HTTP para definir como o cliente pode interagir com os recursos. Cada rota tem uma função específica dentro da API. Por exemplo:

- **`GET /products`**: Retorna uma lista de produtos.
- **`POST /products`**: Cria um novo produto.
- **`GET /products/:id`**: Retorna os detalhes de um produto específico.
- **`PUT /products/:id`**: Atualiza todas as informações de um produto em específico.
- **`PATCH /products/:id`**: Atualização parcial das informações de um produto em específico.
- **`DELETE /products/:id`**: Remove um produto específico.

Ao estruturar rotas, é importante mantê-las consistentes e organizadas para facilitar a manutenção e compreensão da API.

Request Body

O Request Body é a parte de uma requisição HTTP onde você envia os dados necessários para a execução de uma ação em uma API REST. Ele é usado em métodos HTTP como POST, PUT, PATCH, entre outros, onde é preciso fornecer informações detalhadas para a API processar a solicitação.
Imagine que você tem uma API para gerenciar usuários, e você quer criar um novo usuário. O Request Body pode ser assim:

```
POST /users

{
  "name": "João Silva",
  "email": "joao.silva@example.com"
}
```

Request Headers

Os Request Headers são informações adicionais enviadas junto com uma requisição HTTP que fornecem contexto sobre a própria requisição, o cliente, ou o servidor. Eles são usados para informar ao servidor detalhes como o tipo de conteúdo que está sendo enviado, as credenciais de autenticação, o tipo de resposta esperada, e muito mais. Alguns exemplos de Request Headers comuns incluem:

- **Content-Type:** Indica o tipo de dados que está sendo enviado no corpo da requisição (Request Body). É fundamental para que o servidor saiba como interpretar os dados recebidos.
- **Authorization:** Usado para enviar credenciais de autenticação. Pode conter tokens de acesso ou credenciais básicas.
- **Accept:** Especifica o tipo de resposta que o cliente espera do servidor.
- **Host:** Especifica o domínio ou IP do servidor para o qual a requisição está sendo enviada, sendo útil especialmente em ambientes de servidores compartilhados.

```
POST /users
Host: api.example.com
Content-Type: application/json
Authorization: Bearer <seu_token_aqui>
Accept: application/json

{
  "name": "Maria Silva",
  "email": "maria.silva@example.com"
}
```

Path/Route Params

Os Path Params (parâmetros de caminho ou parâmetros de rota) são partes **variáveis** de uma URL que são usadas para **identificar recursos específicos** em uma API REST. Eles fazem parte do endpoint da URL e são incluídos diretamente no caminho (path) da requisição.
É comumente usado quando se deseja buscar os dados de um recurso específico, atualizar ou excluir um recurso. Por exemplo:

- **`GET /users/123`**
  Aqui, o parâmetro de caminho é `123`, que representa o ID do usuário a ser buscado, podendo ser substituído por qualquer outro valor. O servidor usará esse ID para buscar o usuário correspondente.

- **`GET /users/joao`**
  Aqui, o parâmetro de caminho é `joao`, que representa o username do usuário a ser buscado, podendo ser substituído por qualquer outro valor. O servidor usará esse username para buscar o usuário correspondente.

- **`PUT /users/123`**
  Aqui, o parâmetro de caminho é `123`, que representa o ID do usuário a ser atualizado, podendo ser substituído por qualquer outro valor. O servidor usará esse ID para encontrar e aplicar as alterações no registro de usuário correto.

- **`DELETE /users/123`**
  Aqui, o parâmetro de caminho é `123`, que representa o ID do usuário a ser removido, podendo ser substituído por qualquer outro valor. O servidor usará esse ID para encontrar e remover o registro de usuário correto.

Query Params

Os Query Params (Parâmetros de Consulta) são usados para passar informações adicionais na URL de uma requisição HTTP. Eles são normalmente opcionais e, quando presentes na requisição, são utilizados para filtrar, ordenar, ou modificar os dados retornados por uma API sem alterar o recurso principal solicitado.
Os Query Params são passados na URL após o ponto de interrogação (?) e podem ser compostos por um ou mais pares chave-valor, separados por um símbolo de igual (=). Quando há múltiplos parâmetros, eles são separados por um símbolo de &.

Exemplos de URLs com Query Params:

- **`GET /products?category=electronics`**
  Filtrar produtos pela categoria "electronics".

- **`GET /users?age=30&status=active&sort=desc`**
  Retorna usuários com idade 30, status "ativo" e ordenados de forma decrescente.

Status Code

Os Status Codes são códigos de resposta que fazem parte do protocolo HTTP. Eles indicam o resultado da requisição feita pelo cliente (navegador, API, etc.) ao servidor. Cada código é um número de três dígitos e pertence a uma das cinco categorias principais, baseadas no primeiro dígito.

Faixa	Categoria	Descrição
1xx	Informativo	Indica que a requisição foi recebida e está em processamento.
2xx	Sucesso	Indica que a requisição foi bem-sucedida
3xx	Redirecionamento	Indica que é necessário tomar uma ação adicional para completar a requisição.
4xx	Erro do cliente	Indica que houve um erro na requisição feita pelo cliente.
5xx	Erro do servidor	Indica que o servidor encontrou um erro ao processar a requisição.

➔ **1xx – Informativo**

- **100 Continue:** O servidor recebeu os cabeçalhos e o cliente pode continuar a enviar o corpo da requisição.
- **101 Switching Protocols:** Indica que o servidor está mudando o protocolo, conforme solicitado pelo cliente.

➔ **2xx – Sucesso**

- **200 OK:** A requisição foi bem-sucedida. Usado para respostas genéricas.
- **201 Created:** Um novo recurso foi criado com sucesso (geralmente usado em requisições POST).
- **204 No Content:** A requisição foi processada com sucesso, mas não há conteúdo para retornar.

➔ **3xx – Redirecionamento**

- **301 Moved Permanently:** O recurso solicitado foi movido para outra URL permanentemente.

- **302 Found:** O recurso foi temporariamente movido para outra URL.
- **304 Not Modified:** Indica que o recurso não foi modificado e o cliente pode usar a versão em cache.

→ **4xx – Erro do cliente**

- **400 Bad Request:** A requisição não pôde ser processada devido a um erro do cliente (ex.: formato inválido).
- **401 Unauthorized:** A requisição requer autenticação. O cliente não forneceu as credenciais ou elas são inválidas.
- **403 Forbidden:** O cliente não tem permissão para acessar o recurso.
- **404 Not Found:** O recurso solicitado não foi encontrado no servidor.
- **422 Unprocessable Entity:** Os dados fornecidos são válidos em formato, mas não podem ser processados.

→ **5xx – Erro do servidor**

- **500 Internal Server Error:** Um erro genérico ocorreu no servidor.
- **501 Not Implemented:** O servidor não suporta a funcionalidade requerida.
- **502 Bad Gateway:** Um servidor intermediário recebeu uma resposta inválida de outro servidor.
- **503 Service Unavailable:** O servidor não está disponível (sobrecarga ou manutenção).
- **504 Gateway Timeout:** Um servidor intermediário não recebeu uma resposta a tempo do servidor final.

JSON

O JSON (*JavaScript Object Notation*) é um formato leve e amplamente utilizado para troca de dados em APIs RESTful. Ele é baseado em texto e organizado como uma coleção de pares chave-valor, facilitando a comunicação entre cliente e servidor. Sua simplicidade, legibilidade e compatibilidade com várias linguagens de programação o tornam a escolha padrão para APIs modernas. Veja abaixo algumas das motivações para se utilizar JSON em APIs REST.

- **Leve e eficiente:** JSON possui uma sintaxe minimalista, o que reduz o tamanho da mensagem e acelera a transmissão de dados.

- **Legível para humanos:** O formato simples de pares chave-valor é fácil de entender e depurar.

- **Compatibilidade com várias linguagens:** A maioria das linguagens de programação tem bibliotecas nativas ou externas para manipular JSON.

- **Baseado em texto:** Diferente de formatos binários, como XML ou Protobuf, o JSON pode ser visualizado diretamente sem ferramentas especiais.

- **Flexibilidade:** JSON suporta objetos, arrays e valores simples (strings, números, booleanos, nulos), o que permite representar estruturas de dados complexas.

Exemplo de corpo JSON para requisição `POST /users`:

```
{
  "name": "Carlos Souza",
  "email": "carlos.souza@example.com",
  "age": 28
}
```

Exemplo de corpo JSON para resposta à `GET /users`:

```
{
    "ok": true,
    "message": "Usuários listados com sucesso",
    "data": [
        {
            "id": 1,
            "name": "João Silva",
            "email": "joao.silva@example.com",
            "age": 30
        }
    ]
}
```

Conclusão

A arquitetura REST é uma abordagem poderosa e padronizada para o desenvolvimento de APIs. Com base nos conceitos apresentados, é possível criar sistemas eficientes, escaláveis e que seguem boas práticas de desenvolvimento. A compreensão detalhada de cada elemento, como URLs, métodos HTTP, status codes, body, params, query e headers, é essencial para a criação de APIs robustas e de fácil manutenção.

BONS ESTUDOS

growdev